DEPLOYMENT GUIDE

Holdfast

Containerization with Docker Compose and deployment to AWS Lightsail, step by step, with every command used, why it is there, and what to do when a step breaks.

Live app: **https://holdfast-ai.duckdns.org**
Repository: github.com/itzzfalcon52/holdfast
Stack: Caddy · Express + Better Auth · FastAPI + worker · Neon PostgreSQL · NVIDIA Nemotron
Written November 2026 · Contains no secrets: every key and password is a placeholder.

Contents

1. Overview: what was deployed and how it fits together

- Architecture
- Request routing
- Why one server
- What we actually used

2. Containerization: every file explained

- 2.1 .dockerignore: keep secrets and junk out of images
- 2.2 Dockerfile.api: FastAPI and the worker
- 2.3 api-entrypoint.sh: two processes, one container
- 2.4 Dockerfile.backend: the account service
- 2.5 Dockerfile.web: build the React app, serve it with Caddy
- 2.6 Caddyfile: HTTPS, routing, limits, headers
- 2.7 docker-compose.yml: the whole stack
- 2.8 .env.production.example: the settings template

3. Code changes made so the app could run in containers

- 3.1 Backend listens on all interfaces and trusts the proxy
- 3.2 Production refuses insecure origins
- 3.3 Streaming upload limits (large CSVs without running out of memory)
- 3.4 Compose passes connector and email settings; empty means "off"
- 3.5 Connector key generator
- 3.6 Backup script fixes

4. AWS deployment, step by step (as performed)

- Step 1: Create the Lightsail instance
- Step 2: Static IP and firewall
- Step 3: Connect and verify Docker and swap
- Step 4: Read access to the private GitHub repository (deploy key)
- Step 5: Create the server's settings file
- Step 6: Build and start
- Step 7: Create account tables, check health
- Step 8: Open the app and verify
- Step 9: A free, nicer address with DuckDNS

5. Running it: updates, logs, backups, scaling

- 5.1 Deploying a new version
- 5.2 Everyday commands
- 5.3 Backups
- 5.4 Upgrading the server size
- 5.5 Rotating secrets
- 5.6 Costs and housekeeping

6. Alternatives and when to use them

7. Appendix: cheat sheet and troubleshooting index

- Full deploy in one place

- Troubleshooting index
- Verification record

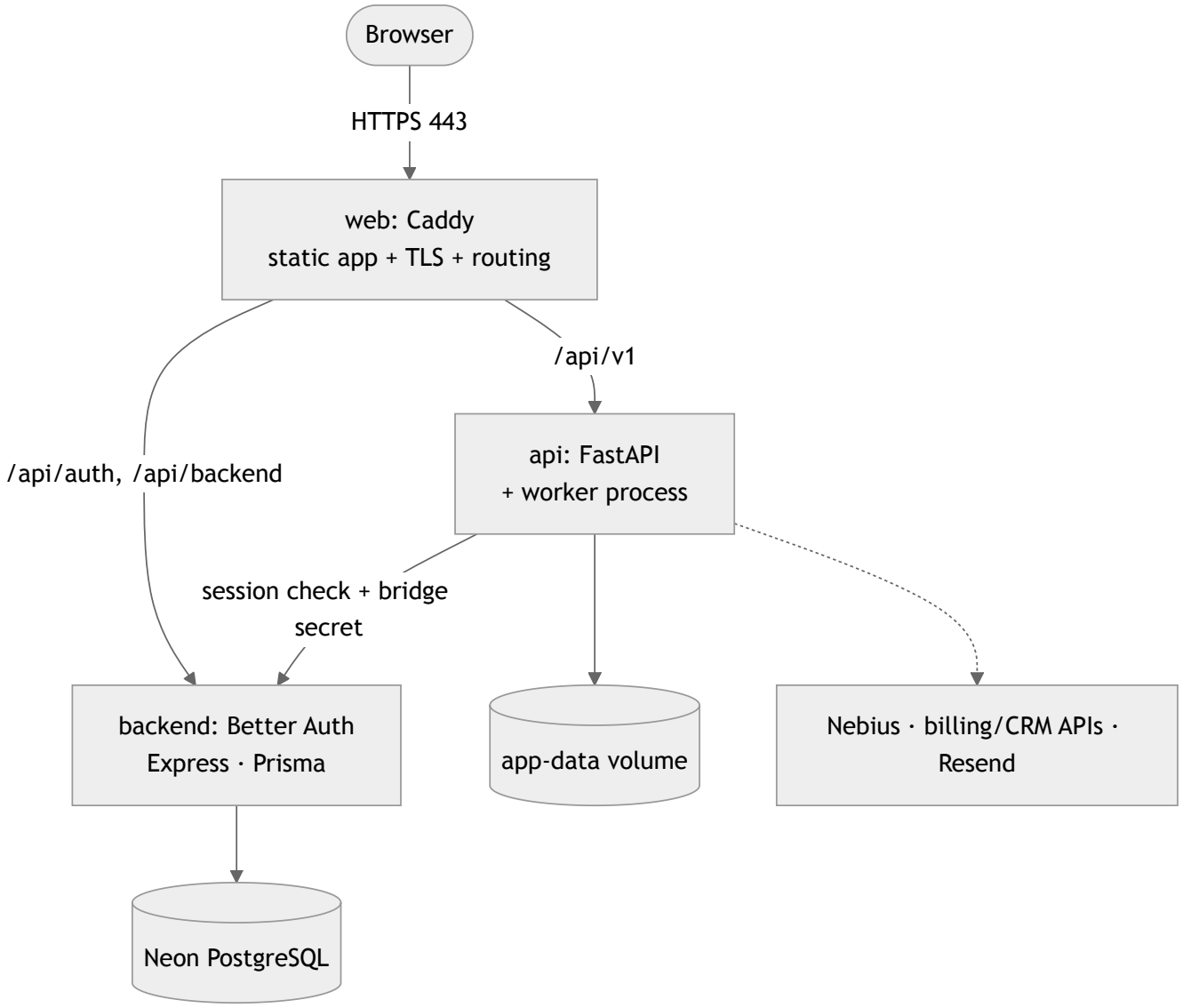
> How to read the boxes: grey blocks are commands or files, copied from the repository or from the session where the deployment was done. Red **If this breaks** boxes list the symptom, the likely cause and the fixes to try, in order.

1. Overview: what was deployed and how it fits together

Holdfast is deployed as **one Linux server running Docker Compose** on AWS Lightsail. Three containers run on it, plus two managed services outside it:

Piece	Runs where	Job
web (Caddy)	Container on the server	Serves the built React app, terminates HTTPS with an automatic Let's Encrypt certificate, and routes `/api/*` to the right service.
backend (Express + Better Auth + Prisma)	Container on the server	Sign-up, sign-in, sessions, organisations, member roles; the private identity endpoint the API asks.
api (FastAPI + worker)	Container on the server	Data, machine learning, scheduler, connectors, AI agent. The worker process runs training, scoring and daily checks in the same container.
app-data volume	Docker volume on the server's disk	Per-workspace SQLite databases, uploaded files, models, scores. Survives container rebuilds.
Neon PostgreSQL	Managed, outside AWS	User accounts, sessions and organisations (same database used in development).
Nebius (NVIDIA Nemotron)	Managed API	Briefs, reasons and Ask AI. Called over HTTPS with your key.

Architecture



Request routing

Browser path	Goes to	Why
`/`, `/product`, `/app/...`	Caddy serves files from `/srv`	The React single-page app; unknown paths fall back to `index.html`.
`/api/auth/*`, `/api/backend/*`	backend:3001	Better Auth endpoints and team/member endpoints.
`/api/v1/*`	api:8000	Everything else: data, models, scores, AI, connectors.

This is the same routing the Vite dev server uses locally, so the browser code needs no changes between development and production. Only ports 22 (SSH), 80 and 443 are open on the server; the backend and API are reachable only through Caddy on the private Docker network.

Why one server

- The API and worker share files and SQLite databases, so they must share a disk.

- One machine is cheap (the 2 GB Lightsail plan is $12/month; AWS lists it with 3 months free and new accounts get credits) and simple to operate.
- When it outgrows one server, the documented path is moving metadata to PostgreSQL and files to object storage, then running more API and worker containers.

What we actually used

Setting	Value
Provider / service	AWS Lightsail
Region	Sydney, Zone A (ap-southeast-2a): the account would not open Mumbai
Image	Linux operating system → Ubuntu 24.04 LTS (OS only)
Plan	$12/month: 2 GB RAM, 2 vCPUs, 60 GB SSD, plus 4 GB swap
Static IP	52.64.200.202 (name `holdfast-ip`)
Firewall	SSH 22, HTTP 80, HTTPS 443
Address	First `52-64-200-202.sslip.io`, then `holdfast-ai.duckdns.org` (free DuckDNS subdomain)
Code	Private GitHub repo cloned with a read-only deploy key
Result	All three images built in about 4½ minutes on the first try; health check OK

2. Containerization: every file explained

All deployment files live in `deploy/`. Builds run from the repository root as the build context (`context: ..` in compose), so Dockerfiles can copy from `apps/`.

2.1 `.dockerignore`: keep secrets and junk out of images

.dockerignore

```
# Keep build contexts small and free of secrets and customer data.
**/node_modules
**/.venv
**/dist
**/__pycache__
.data
.pnpm-store
.tooling
.turbo
**/.env
**/.env.*
!**/.env.example
!**/.env.*.example
deploy/.env
.git
```

- `**/.env`, `**/.env.*` and `deploy/.env` are excluded, so no secret can be baked into an image. The two `!` lines re-include the `.example` templates (placeholders only).
- `.data` (local customer files and models), virtual environments, `node_modules` and build outputs are excluded so the build context stays small and builds are reproducible.

2.2 `Dockerfile.api`: FastAPI and the worker

deploy/Dockerfile.api

```
# FastAPI and the training/monitoring worker share one container because they share /data.
FROM python:3.13-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1
# libgomp1: OpenMP runtime for XGBoost. tini: forwards signals to both processes.
RUN apt-get update && apt-get install -y --no-install-recommends libgomp1 tini \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY apps/api/requirements.txt .
RUN pip install --require-hashes -r requirements.txt
COPY apps/api/pyproject.toml apps/api/README.md* ./
COPY apps/api/src ./src
RUN pip install --no-deps .
COPY deploy/api-entrypoint.sh /usr/local/bin/api-entrypoint
RUN chmod +x /usr/local/bin/api-entrypoint \
    && useradd --create-home --uid 10001 app && mkdir -p /data && chown app /data
USER app
ENV CI_DATA_DIRECTORY=/data CI_ENVIRONMENT=production CI_AUTH_MODE=account \
    CI_AUTH_SERVICE_URL=http://backend:3001
EXPOSE 8000
ENTRYPOINT ["tini", "--", "api-entrypoint"]
```

Line	What it does and why
`FROM python:3.13-slim`	Small Debian image with the exact Python version the project pins.
`ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1`	No `.pyc` clutter, logs appear immediately in `docker compose logs`, smaller image.
`apt-get install libgomp1 tini`	`libgomp1` is the OpenMP runtime XGBoost needs (on macOS this was `brew install libomp`). `tini` is a tiny init that forwards stop signals to both processes and reaps zombies.
`COPY apps/api/requirements.txt` then `pip install --require-hashes`	Dependencies install first so Docker caches this slow layer; `--require-hashes` refuses any package whose hash differs from the lock (supply-chain safety).
`COPY ... src` then `pip install --no-deps .`	Installs our own package without touching the pinned dependencies.
`useradd --uid 10001 app`, `chown app /data`, `USER app`	The app never runs as root; it can only write to `/data`.
`ENV CI_DATA_DIRECTORY=/data CI_ENVIRONMENT=production CI_AUTH_MODE=account CI_AUTH_SERVICE_URL=http://backend:3001`	Data goes to the mounted volume; account mode means every request is checked against the backend over the private network (`backend` is the compose service name).
`ENTRYPOINT ["tini", "--", "api-entrypoint"]`	Starts both processes through tini (next file).

2.3 `api-entrypoint.sh`: two processes, one container

deploy/api-entrypoint.sh

```
#!/bin/bash
# Runs the API and the worker together; if either exits, the container exits and Docker restarts
it.
set -e
uvicorn customer_intelligence.main:app --host 0.0.0.0 --port 8000 --proxy-headers \
  --forwarded-allow-ips='*' &
python -m customer_intelligence.worker &
wait -n
exit 1
```

The API and the worker must share the same `/data` volume and SQLite files, so they run in one container. Both start in the background; `wait -n` waits for *either* to exit, then the script exits with an error so Docker's `restart: unless-stopped` restarts the whole container. `--proxy-headers` lets uvicorn read the real client scheme/address from Caddy.

2.4 `Dockerfile.backend`: the account service

deploy/Dockerfile.backend

```
# Better Auth + Prisma account service. Dev dependencies stay so `prisma migrate deploy` can run.
FROM node:22-slim
WORKDIR /repo
RUN apt-get update && apt-get install -y --no-install-recommends openssl ca-certificates \
    && rm -rf /var/lib/apt/lists/*
RUN corepack enable && corepack prepare pnpm@11.25.0 --activate
COPY pnpm-lock.yaml pnpm-workspace.yaml package.json ./
COPY apps/backend/package.json apps/backend/
RUN pnpm install --frozen-lockfile --filter @ci/backend...
COPY apps/backend apps/backend
RUN pnpm --filter @ci/backend build
WORKDIR /repo/apps/backend
ENV NODE_ENV=production HOST=0.0.0.0 PORT=3001 TRUST_PROXY=true
USER node
EXPOSE 3001
CMD ["node", "dist/server.js"]
```

Line	What it does and why
`FROM node:22-slim`	Node 22 (the project needs 22.12+).
`apt-get install openssl ca-certificates`	Prisma's query engine needs OpenSSL; CA certificates let it reach Neon over TLS.
`corepack prepare pnpm@11.25.0 --activate`	Exactly the pnpm version pinned in `package.json` (`packageManager`), so the lockfile is read identically.
`COPY pnpm-lock.yaml pnpm-workspace.yaml package.json` + `apps/backend/package.json`	Only manifests first, so dependency installs are cached until they change.
`pnpm install --frozen-lockfile --filter @ci/backend...`	Installs only the backend and what it depends on; fails instead of silently changing the lockfile.
`pnpm --filter @ci/backend build`	Generates the Prisma client and compiles TypeScript to `dist/`.
`ENV NODE_ENV=production HOST=0.0.0.0 PORT=3001 TRUST_PROXY=true`	Production mode turns on secure cookies and demands an HTTPS `APP_ORIGIN`; `HOST=0.0.0.0` listens on the container network (locally it is `127.0.0.1`); `TRUST_PROXY` uses the client address Caddy writes for sign-in rate limits.
`USER node`, `CMD ["node", "dist/server.js"]`	Runs unprivileged. Dev dependencies are kept so `prisma migrate deploy` can run from this image.

2.5 `Dockerfile.web`: build the React app, serve it with Caddy

deploy/Dockerfile.web

```
# Builds the React app and serves it with Caddy, which is also the HTTPS reverse proxy.
FROM node:22-slim AS build
WORKDIR /repo
RUN corepack enable && corepack prepare pnpm@11.25.0 --activate
COPY pnpm-lock.yaml pnpm-workspace.yaml package.json ./
COPY apps/web/package.json apps/web/
RUN pnpm install --frozen-lockfile --filter @ci/web...
COPY apps/web apps/web
RUN pnpm --filter @ci/web build

FROM caddy:2.10-alpine
COPY --from=build /repo/apps/web/dist /srv
COPY deploy/Caddyfile /etc/caddy/Caddyfile
```

A **two-stage build**: stage one installs the web workspace and runs `vite build`; stage two starts from the small official `caddy:2.10-alpine` image and copies only the built `dist/` files and the Caddyfile. Node and source code never reach the final image.

2.6 `Caddyfile`: HTTPS, routing, limits, headers

deploy/Caddyfile

```
# {$DOMAIN} comes from .env. Caddy gets and renews the HTTPS certificate automatically.
{$DOMAIN} {
        encode zstd gzip
        request_body {
                max_size {$MAX_REQUEST_MB:820}MB
        }

        # Same routing as the Vite dev proxy.
        handle /api/auth/* {
                reverse_proxy backend:3001
        }
        handle /api/backend/* {
                reverse_proxy backend:3001
        }
        handle /api/v1/* {
                reverse_proxy api:8000 {
                        # Large uploads and long AI answers.
                        transport http {
                                read_timeout 10m
                                write_timeout 10m
                        }
                }
        }
        handle {
                root * /srv
                try_files {path} /index.html
                file_server
        }

        header {
                Strict-Transport-Security "max-age=31536000; includeSubDomains"
                X-Content-Type-Options nosniff
                Referrer-Policy strict-origin-when-cross-origin
                -Server
        }
}
```

- `{$DOMAIN}` comes from the environment. Because it is a real hostname, Caddy automatically obtains and renews a Let's Encrypt certificate (it needs ports 80 and 443 reachable and DNS pointing at the server).
- `request_body max_size` allows large CSV/Excel uploads (the API applies its own per-path limits, 200 MB by default).
- `read_timeout/write_timeout 10m` for `/api/v1/*`: large uploads and long streamed AI answers.
- `try_files {path} /index.html`: deep links such as `/app/renewals` load the React app.
- Security headers: HSTS, `nosniff`, a strict referrer policy, and the `Server` header removed.

2.7 `docker-compose.yml`: the whole stack

deploy/docker-compose.yml

```
# One server: Caddy (web + HTTPS), the account backend, and the API with its worker.
# Run from the repository root:  docker compose -f deploy/docker-compose.yml --env-file
deploy/.env up -d --build
name: customer-intelligence

services:
  web:
    build: { context: .., dockerfile: deploy/Dockerfile.web }
    restart: unless-stopped
    ports: ['80:80', '443:443']
    environment:
      DOMAIN: ${DOMAIN}
      MAX_REQUEST_MB: ${MAX_REQUEST_MB:-820}
    volumes:
      - caddy-data:/data
      - caddy-config:/config
    depends_on: [backend, api]

  backend:
    build: { context: .., dockerfile: deploy/Dockerfile.backend }
    restart: unless-stopped
    environment:
      DATABASE_URL: ${DATABASE_URL}
      DIRECT_DATABASE_URL: ${DIRECT_DATABASE_URL:-}
      BETTER_AUTH_SECRET: ${BETTER_AUTH_SECRET}
      INTERNAL_API_SECRET: ${INTERNAL_API_SECRET}
      APP_ORIGIN: https://${DOMAIN}

  api:
    build: { context: .., dockerfile: deploy/Dockerfile.api }
    restart: unless-stopped
    environment:
      APP_ORIGIN: https://${DOMAIN}
      INTERNAL_API_SECRET: ${INTERNAL_API_SECRET}
      CI_ALLOWED_HOSTS: '["${DOMAIN}","api","localhost"]'
      NEBIUS_API_KEY: ${NEBIUS_API_KEY}
      NEBIUS_BASE_URL: ${NEBIUS_BASE_URL}
      NEBIUS_MODEL: ${NEBIUS_MODEL}
      CI_MAX_UPLOAD_MB: ${CI_MAX_UPLOAD_MB:-200}
      CI_MAX_ROWS: ${CI_MAX_ROWS:-1000000}
      CI_WORKER_TIMEOUT_SECONDS: ${CI_WORKER_TIMEOUT_SECONDS:-1800}
      # Connectors (encryption key for stored credentials) and email digests; empty = off.
      CI_CREDENTIALS_KEY: ${CI_CREDENTIALS_KEY:-}
      CI_RESEND_API_KEY: ${CI_RESEND_API_KEY:-}
      CI_EMAIL_FROM: ${CI_EMAIL_FROM:-}
    volumes:
      - app-data:/data
    healthcheck:
      test:
        [
          'CMD',
          'python',
          '-c',
          "import urllib.request;urllib.request.urlopen('http://localhost:8000/api/v1/health')",
        ]
      interval: 30s
      timeout: 5s
```

```
      retries: 3

volumes:
  app-data: # customer files, models, scores and per-account SQLite workspaces
  caddy-data: # HTTPS certificates
  caddy-config:
```

- **name: customer-intelligence**: fixes the project name, so the volume is always `customer-intelligence_app-data` (the backup script relies on it).
- **web** is the only service with published ports (80, 443). `caddy-data` keeps certificates across rebuilds (avoids Let's Encrypt rate limits).
- **backend** and **api** receive secrets from `deploy/.env` through `${...}` substitution. `APP_ORIGIN: https://${DOMAIN}` is built from the domain, so changing `DOMAIN` updates both services.
- `CI_ALLOWED_HOSTS` lets the API accept requests for the public domain and for internal names.
- `${CI_CREDENTIALS_KEY:-}`-style defaults mean "empty if not set"; the API treats empty as "off" (section 3).
- The api **healthcheck** calls `/api/v1/health` every 30 s; `docker compose ps` shows `(healthy)`.
- **volumes**: named Docker volumes live under `/var/lib/docker/volumes` and survive `up --build`, restarts and reboots.

2.8 `.env.production.example`: the settings template

deploy/.env.production.example

```
# Copy to deploy/.env on the server (never commit it). Placeholders only.
DOMAIN=app.example.com

# Neon (same project as development, or a separate production branch). Keep sslmode=require.
DATABASE_URL=postgresql://user:password@host/db?sslmode=require
DIRECT_DATABASE_URL=

# Generate each with: openssl rand -hex 32
BETTER_AUTH_SECRET=
INTERNAL_API_SECRET=

# Nebius AI (Nemotron)
NEBIUS_API_KEY=
NEBIUS_BASE_URL=https://api.tokenfactory.nebius.com/v1/
NEBIUS_MODEL=nvidia/Nemotron-3_5-Lightning

# Connectors: encrypts stored Razorpay/Zoho/Stripe/HubSpot credentials. Generate once and keep it
# (changing it makes saved connections unreadable):
#   openssl rand -base64 32 | tr '+/' '-_'
CI_CREDENTIALS_KEY=

# Email digests through Resend (optional; both needed). Sender on a domain verified in Resend.
CI_RESEND_API_KEY=
CI_EMAIL_FROM=

# Upload limits and training time budget
CI_MAX_UPLOAD_MB=200
CI_MAX_ROWS=1000000
CI_WORKER_TIMEOUT_SECONDS=1800

# Backups (optional): any S3-compatible bucket, e.g. Cloudflare R2 or AWS S3
BACKUP_BUCKET=
BACKUP_ENDPOINT=
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
```

Variable	How to get or generate it
`DOMAIN`	Your hostname without `https://`, e.g. `holdfast-ai.duckdns.org`.
`DATABASE_URL`, `DIRECT_DATABASE_URL`	From Neon (same values as `apps/backend/.env` on your Mac). Keep `?sslmode=require`.
`BETTER_AUTH_SECRET`, `INTERNAL_API_SECRET`	`openssl rand -hex 32` (one each). Compose passes the same `INTERNAL_API_SECRET` to both services.
`NEBIUS_API_KEY`, `NEBIUS_BASE_URL`, `NEBIUS_MODEL`	From `apps/api/.env`; defaults point at Nebius and Nemotron.
`CI_CREDENTIALS_KEY`	`openssl rand -base64 32 | tr '+/' '-_'`: a Fernet key. Never change it once connectors are saved.
`CI_RESEND_API_KEY`, `CI_EMAIL_FROM`	Optional, from Resend. Empty = email off.

Variable	How to get or generate it
`BACKUP_*`, `AWS_*`	Optional, for `backup.sh` to S3/R2.

3. Code changes made so the app could run in containers

Containerizing exposed a few places where the app assumed it ran on a laptop. These are the changes, with the code.

3.1 Backend listens on all interfaces and trusts the proxy

Locally the backend listens on `127.0.0.1`, which is unreachable from another container. Two settings were added and the server listens on `config.HOST`:

backend config and server

```
// apps/backend/src/config.ts (schema excerpt)
PORT: z.coerce.number().int().min(1024).max(65535).default(3001),
// Loopback by default; a container sets 0.0.0.0 and stays unpublished behind the proxy.
HOST: z.string().default('127.0.0.1'),
// Behind Caddy only: trust the client address the proxy writes into X-Forwarded-For.
TRUST_PROXY: z.enum(['true', 'false']).default('false').transform((v) => v === 'true'),

// apps/backend/src/server.ts
const server = app.listen(config.PORT, config.HOST, () =>
  console.log(`Account backend ready at http://${config.HOST}:${config.PORT}`));
```

With `TRUST_PROXY=true` (set in the Dockerfile) the sign-in rate limiter keys on the last `X-Forwarded-For` entry, which Caddy writes and clients cannot spoof through it.

3.2 Production refuses insecure origins

apps/backend/src/config.ts

```
if (parsed.data.NODE_ENV === 'production' && !parsed.data.APP_ORIGIN.startsWith('https://')) {
  throw new Error('Production APP_ORIGIN must use HTTPS.');
}
```

Compose always sets `APP_ORIGIN=https://${DOMAIN}`, so this holds. In development only, the backend also accepts the `localhost` / `127.0.0.1` twin (the `allowedOrigins` helper) so sign-in works from either address.

3.3 Streaming upload limits (large CSVs without running out of memory)

The old body-limit middleware buffered whole requests. It was rewritten to stream: a declared `Content-Length` over the limit is refused before reading; chunked bodies are counted as they arrive. Uploads stream to a `.part` file. Limits are configurable:

Variable	Default	Meaning
`CI_MAX_UPLOAD_MB`	200	Largest CSV upload
`CI_MAX_ROWS`	1,000,000	Largest table
`CI_MAX_COLUMNS`	500	Widest table
`CI_MAX_EXCEL_MB`	50	Largest Excel file
`CI_WORKER_TIMEOUT_SECONDS`	1800	Training time limit (max 4 h)

3.4 Compose passes connector and email settings; empty means "off"

The first compose file did not pass the connector encryption key or Resend settings to the API. They were added, and a validator makes an empty value mean "not configured" instead of an invalid key:

deploy/docker-compose.yml (api environment)

```
      # Connectors (encryption key for stored credentials) and email digests; empty = off.
      CI_CREDENTIALS_KEY: ${CI_CREDENTIALS_KEY:-}
      CI_RESEND_API_KEY: ${CI_RESEND_API_KEY:-}
      CI_EMAIL_FROM: ${CI_EMAIL_FROM:-}
```

apps/api/src/customer_intelligence/core/config.py

```
@field_validator("credentials_key", "resend_api_key", "email_from", mode="before")
@classmethod
def blank_is_unset(cls, value: Any) -> Any:
    # Deployment templates pass optional settings as empty strings; empty means "off".
    return None if isinstance(value, str) and not value.strip() else value
```

3.5 Connector key generator

Locally `pnpm setup:connectors` (`scripts/setup-connectors.mjs`) writes a Fernet key to `apps/api/.env` once and never prints it. On the server the same kind of key is made with `openssl rand -base64 32 | tr '+/' '-_'` (32 random bytes, URL-safe base64, which is exactly the Fernet format; this was verified against the `cryptography` library).

3.6 Backup script fixes

deploy/backup.sh

```
# Export the settings so the aws command sees the backup credentials.
set -a
. ./.env
set +a
```

Two bugs were fixed before first use: the script loaded `.env` without exporting the variables (the AWS CLI would not have seen the keys), and Ubuntu 24.04 has no `awscli` apt package, so installation is now `sudo snap install aws-cli --classic`.

4. AWS deployment, step by step (as performed)

These are the steps exactly as performed, in order. Each ends with what you should see and an **If this breaks** box.

Step 1: Create the Lightsail instance

1. AWS console → search **Lightsail** → **Create instance**.
2. **Region:** the new Lightsail console has no picker on this page; the region is the `?region=` part of the address bar. To use Mumbai, change it to `?region=ap-south-1`. Our account stayed in Sydney (`ap-southeast-2`), which works fine.
3. **Platform:** *Linux operating system* (not "Linux apps": those preinstall WordPress, OpenClaw, etc.). **Blueprint:** *Ubuntu 24.04 LTS*.
4. **Plan:** *$12/month, 2 GB RAM*. The $5 plan (0.5–1 GB) cannot build or run the stack.
5. **Advanced settings** → **Add launch script** (runs once as root at first boot):

Lightsail launch script

```
#!/bin/bash
fallocate -l 4G /swapfile && chmod 600 /swapfile && mkswap /swapfile && swapon /swapfile
echo '/swapfile none swap sw 0 0' >> /etc/fstab
curl -fsSL https://get.docker.com | sh
usermod -aG docker ubuntu
```

- Lines 1–2 create a 4 GB swap file and make it permanent, so builds and training don't run out of memory on 2 GB.
- Line 3 installs Docker Engine and the Compose plugin with Docker's official script.
- Line 4 lets the `ubuntu` user run Docker without `sudo`.

6. **Name:** `holdfast-backend`. SSH key: leave *Default*. Automatic snapshots: off for now (turn on later, see Operations). **Create instance**.

IF THIS BREAKS

Symptom: The $5 plan is the only one available
Likely cause: A "Linux apps" blueprint with higher minimum size is selected.

1. Choose *Linux operating system* → *Ubuntu 24.04*; all plans appear.
2. Don't use the $5 plan for this app; pick 2 GB or 4 GB.

Symptom: Changing `?region=` errors or bounces back
Likely cause: Managed/sandbox accounts are often limited to one region.

1. Stay in the allowed region; latency difference from India to Sydney is about 120–150 ms.
2. Use your own AWS account if you need Mumbai.

Symptom: Launch script was forgotten
Likely cause: Nothing to fix later; run it manually (Step 3).

1. Run the same lines with `sudo` in the SSH terminal.

Step 2: Static IP and firewall

1. Instance → **Networking** tab → **Attach static IP** → name `holdfast-ip` → attach. You get a new IP (ours: **52.64.200.202**). It is free while attached and never changes on restart.
2. **IPv4 Firewall** → **+ Add rule** → **HTTPS**. You should see SSH 22, HTTP 80, HTTPS 443.

IF THIS BREAKS

Symptom: Site unreachable later on HTTPS
Likely cause: Port 443 not open, or rule added only for IPv6.

1. Networking tab → check the HTTPS rule exists under *IPv4* firewall.
2. Also keep HTTP 80 open: Let's Encrypt validates over it.

Symptom: IP changed after a reboot
Likely cause: Static IP not attached (public IP is dynamic).

1. Attach a static IP, then update DuckDNS/DNS to the new address.

Step 3: Connect and verify Docker and swap

Connect tab → **Connect using SSH** (browser terminal; no key file needed). Then:

```
server
```

```
docker --version && free -h
```

Expected: a Docker version (ours: 29.9.2) and `Swap: 4.0Gi`.

IF THIS BREAKS

Symptom: `docker: command not found`
Likely cause: Launch script still running, or not added.

1. Wait 2 minutes and retry.
2. Install manually: `curl -fsSL https://get.docker.com | sudo sh` and add swap with the four swap lines prefixed by `sudo`.

Symptom: `permission denied while trying to connect to the Docker daemon socket`
Likely cause: Your login session predates being added to the `docker` group.

1. Close the SSH window and reconnect.
2. Or run `newgrp docker` in the current session.
3. Or run `sudo usermod -aG docker ubuntu` if the group was never added.

Symptom: Swap shows 0B
Likely cause: Swap file not created.

1. `sudo fallocate -l 4G /swapfile && sudo chmod 600 /swapfile && sudo mkswap /swapfile && sudo swapon /swapfile`
2. Add `/swapfile none swap sw 0 0` to `/etc/fstab` so it survives reboots.

Symptom: "Connect using SSH" button is grey
Likely cause: Status checks still initializing.

1. Wait 1–3 minutes and refresh the page.

Step 4: Read access to the private GitHub repository (deploy key)

server

```
docker ps && ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519 && cat ~/.ssh/id_ed25519.pub
```

- `docker ps` confirms Docker works without sudo.
- `ssh-keygen` makes a key pair on the server with no passphrase; `cat` prints the **public** half (safe to copy).

GitHub → repository → **Settings** → **Deploy keys** → **Add deploy key**: title `lightsail-holdfast`, paste the line, **leave "Allow write access" unticked** (the server can read, never push).

server

```
ssh -o StrictHostKeyChecking=accept-new -T git@github.com
git clone git@github.com:itzzfalcon52/customer-intelligence.git && cd customer-intelligence &&
git log -1 --oneline
```

Expected: "*Hi itzzfalcon52/customer-intelligence! You've successfully authenticated…*" and the latest commit. (The repo was later renamed to `holdfast`; GitHub forwards the old name. To switch: `git remote set-url origin git@github.com:itzzfalcon52/holdfast.git`.)

IF THIS BREAKS

Symptom: `Permission denied (publickey)`
Likely cause: Deploy key not added, added to the wrong repo, or a different key file is used.

1. Re-run `cat ~/.ssh/id_ed25519.pub` and compare with the key in GitHub → Settings → Deploy keys.
2. Add it to *this repository*, not your account's SSH keys (either works, but repo deploy keys are safer).
3. Test with `ssh -vT git@github.com` to see which key is offered.

Symptom: `destination path 'customer-intelligence' already exists`
Likely cause: You already cloned it (e.g. after reconnecting).

1. Just `cd ~/customer-intelligence`; nothing to redo.
2. To start fresh: `rm -rf ~/customer-intelligence` then clone (this deletes `deploy/.env` too).

Symptom: `Host key verification failed`
Likely cause: GitHub's host key not trusted yet.

1. Use the `-o StrictHostKeyChecking=accept-new` line above once.

Symptom: Public repo alternative
Likely cause:

1. If the repo is public, skip deploy keys: `git clone https://github.com/itzzfalcon52/holdfast.git`.

Step 5: Create the server's settings file

Important: after any reconnect, start with `cd ~/customer-intelligence`; the terminal opens in your home folder.

server

```
cp deploy/.env.production.example deploy/.env && chmod 600 deploy/.env
sed -i "s|^DOMAIN=.*|DOMAIN=52-64-200-202.sslip.io|" deploy/.env
sed -i "s|^BETTER_AUTH_SECRET=.*|BETTER_AUTH_SECRET=$(openssl rand -hex 32)|" deploy/.env
sed -i "s|^INTERNAL_API_SECRET=.*|INTERNAL_API_SECRET=$(openssl rand -hex 32)|" deploy/.env
K=$(openssl rand -base64 32 | tr '+/' '-_'); sed -i
"s|^CI_CREDENTIALS_KEY=.*|CI_CREDENTIALS_KEY=$K|" deploy/.env; unset K
```

- Copies the template and restricts it to your user (`chmod 600`).
- Each `sed -i "s|^NAME=.*|NAME=value|"` replaces one line in place; `|` is used as the separator because values can contain `/`.
- Secrets are generated *on the server* and never printed.

Then paste the values you can only get from your Mac (`apps/backend/.env` and `apps/api/.env`):

server

```
nano deploy/.env
```

Fill `DATABASE_URL` (and `DIRECT_DATABASE_URL` if you have one) and `NEBIUS_API_KEY`. In the browser terminal use the **Paste into terminal** button. Save with **Ctrl+O, Enter**, exit with **Ctrl+X**. Check which values are set without revealing them:

server

```
sed -E 's/=(.+)/=<set>/' deploy/.env | grep -v '^#' | grep .
```

IF THIS BREAKS

Symptom: `sed: can't read deploy/.env: No such file or directory`
Likely cause: You're in the home folder, not the project.

1. `cd ~/customer-intelligence` and run again.
2. If the file truly doesn't exist, repeat this step from the `cp` line.

Symptom: Neon URL got split or mangled
Likely cause: Pasted across lines, or edited with `sed` (Neon URLs contain `&` and `?`, which `sed` treats specially).

1. Always paste database URLs in `nano`, on one line.
2. Check with the `<set>` command; open `nano` again to fix.

Symptom: You reused the Mac's secrets instead of generated ones
Likely cause: Allowed; works.

1. Better practice: production-only secrets. Changing `BETTER_AUTH_SECRET` later signs everyone out; changing `INTERNAL_API_SECRET` is harmless (both services read the same file).

Step 6: Build and start

```
server
docker compose -f deploy/docker-compose.yml --env-file deploy/.env up -d --build
```

`up` creates the network, volumes and containers; `-d` runs them in the background; `--build` builds the three images first. Ours took about 4½ minutes and ended with *Container … api-1 Started, backend-1 Started, web-1 Started*.

IF THIS BREAKS

Symptom: Build stops with `Killed` or exit code 137
Likely cause: Out of memory during `vite build` / `pnpm install`.

1. Confirm swap: `free -h`.
2. Build one service at a time: `docker compose -f deploy/docker-compose.yml --env-file deploy/.env build web`, then `backend`, then `api`, then `up -d`.
3. Free space and cache: `docker builder prune -f`.
4. Upgrade to the 4 GB plan (Lightsail → Change plan, or snapshot → new instance).

Symptom: `ERR_PNPM_OUTDATED_LOCKFILE` / frozen lockfile error
Likely cause: `pnpm-lock.yaml` doesn't match a `package.json`.

1. On the Mac: `pnpm install`, commit the lockfile, push; on the server: `git pull`, rebuild.

Symptom: pip: `THESE PACKAGES DO NOT MATCH THE HASHES`
Likely cause: `requirements.txt` out of sync with `uv.lock`, or a corrupted download.

1. Retry the build (network glitch).
2. On the Mac re-export requirements with uv (see project docs), commit, push, pull, rebuild.

Symptom: `no space left on device`
Likely cause: Old images and build cache fill the disk.

1. `docker system df` to see usage; `docker image prune -f` and `docker builder prune -f`.
2. Never use `docker volume prune` casually: it can delete `app-data`.

Symptom: Browser SSH disconnected mid-build
Likely cause: Session timeout.

1. Reconnect, `cd ~/customer-intelligence`, run the same command; cached layers make it resume quickly.
2. For long builds use `tmux`: `tmux new -s build`, run the command, detach with Ctrl+B then D.

Symptom: `Bind for 0.0.0.0:80 failed: port is already allocated`
Likely cause: Another web server (e.g. from a 'Linux apps' blueprint) uses port 80/443.

1. `sudo ss -ltnp | grep -E ':80|:443'` to find it; stop/disable it (`sudo systemctl disable --now nginx` or `apache2`).

Step 7: Create account tables, check health

server

```
docker compose -f deploy/docker-compose.yml --env-file deploy/.env run --rm backend
./node_modules/.bin/prisma migrate deploy
docker compose -f deploy/docker-compose.yml --env-file deploy/.env ps
curl -s https://52-64-200-202.sslip.io/api/v1/health; echo
```

- `run --rm backend …` starts a one-off backend container and applies Prisma migrations to Neon. Because we reused the development database it reported nothing pending.
- `ps` should list api (eventually `healthy`), backend and web as `Up`.
- The curl should print `{"status":"ok",...}`. The first HTTPS request can take ~30–60 s while Caddy obtains the certificate.

IF THIS BREAKS

Symptom: Migrate: `P1001 Can't reach database server`
Likely cause: Wrong `DATABASE_URL`, missing `sslmode=require`, or Neon project paused.

1. Re-check the value in `nano deploy/.env` (one line, no quotes needed).
2. Open the Neon console; wake/resume the project.
3. If Neon gives a pooled URL (`-pooler`) put the direct URL in `DIRECT_DATABASE_URL`; migrations need a direct connection.

Symptom: Backend keeps restarting; logs say *Configure apps/backend/.env: DATABASE_URL, BETTER_AUTH_SECRET*
Likely cause: A required value is empty or too short (secret must be ≥ 32 characters).

1. Fill it, then `docker compose -f deploy/docker-compose.yml --env-file deploy/.env up -d`.

Symptom: Backend logs: *Production APP_ORIGIN must use HTTPS*
Likely cause: `DOMAIN` empty, so `APP_ORIGIN` became `https://`.

1. Set `DOMAIN` and `up -d`.

Symptom: curl: `SSL routines … internal error` or connection refused
Likely cause: Certificate not issued yet; DNS not pointing here; ports closed.

1. Wait 60 s and retry.
2. `docker compose -f deploy/docker-compose.yml --env-file deploy/.env logs --tail=60 web` and look for *obtaining certificate* errors.
3. Check DNS: `getent hosts <domain>` must print your static IP.
4. Check firewall: ports 80 and 443 open.

Symptom: Caddy log mentions `rateLimited`
Likely cause: Shared free domains (sslip.io, duckdns.org) share Let's Encrypt limits with everyone.

1. Wait and let Caddy retry (it backs off automatically).
2. Keep the `caddy-data` volume: it stores the certificate so restarts don't request new ones.
3. Long term: your own domain.

Symptom: `502 Bad Gateway` on `/api/v1/*`
Likely cause: API container crashed or still starting.

1. `docker compose -f deploy/docker-compose.yml --env-file deploy/.env logs --tail=80 api`.
2. Common causes: invalid env value (e.g. a malformed `CI_CREDENTIALS_KEY`), out of memory (check `dmesg | tail`).

Step 8: Open the app and verify

1. Browse to the HTTPS address, sign in or sign up (existing accounts work because Neon is shared).
2. **Settings** → **Service status**: ML API connected, worker online.
3. **Check AI connection** confirms Nemotron is reachable from the server.

IF THIS BREAKS

Symptom: Sign-in says it worked but you are bounced back to the login page
Likely cause: Secure cookies need HTTPS on the exact domain in `APP_ORIGIN`.

1. Always use `https://` and the exact `DOMAIN` (no IP, no other hostname).
2. After changing `DOMAIN`, sign in again: old cookies belong to the old host.

Symptom: API returns 503 *Account service is unavailable*
Likely cause: The API cannot verify sessions with the backend.

1. `INTERNAL_API_SECRET` must be identical for both (it is, if both read the same `deploy/.env`).
2. Backend running? `docker compose -f deploy/docker-compose.yml --env-file deploy/.env ps` and its logs.

Symptom: 403 *Request origin is not allowed*
Likely cause: The page's origin differs from `APP_ORIGIN`.

1. Use the exact HTTPS domain; check `DOMAIN` has no `https://` prefix or trailing slash.

Symptom: Worker shows offline
Likely cause: The worker process exited; the entrypoint restarts the container when either process exits.

1. `docker compose -f deploy/docker-compose.yml --env-file deploy/.env logs --tail=80 api`; `docker compose -f deploy/docker-compose.yml --env-file deploy/.env restart api`.

Symptom: AI check fails
Likely cause: Missing or wrong Nebius key/model, or provider outage.

1. Re-check `NEBIUS_*` in `deploy/.env`, then `up -d`.
2. The rest of the app works without AI.

Step 9: A free, nicer address with DuckDNS

sslip.io maps `52-64-200-202.sslip.io` to the IP automatically. To get `holdfast-ai.duckdns.org`:

1. duckdns.org → sign in with GitHub → add sub-domain `holdfast-ai`.
2. DuckDNS fills in the IP of *your* network: replace it with the server's static IP **52.64.200.202** and click **update ip**. Leave IPv6 empty (the firewall only allows HTTPS over IPv4).
3. Treat the DuckDNS **token** like a password: anyone with it can repoint your name.

server

```
cd ~/customer-intelligence
getent hosts holdfast-ai.duckdns.org          # must print 52.64.200.202
sed -i "s|^DOMAIN=.*|DOMAIN=holdfast-ai.duckdns.org|" deploy/.env
docker compose -f deploy/docker-compose.yml --env-file deploy/.env up -d
sleep 20; curl -s https://holdfast-ai.duckdns.org/api/v1/health; echo
```

`up -d` (without `--build`) recreates only the containers whose settings changed; it takes seconds. Caddy gets a certificate for the new name; the old sslip.io address stops working.

IF THIS BREAKS

Symptom: `getent hosts` prints the old IP or nothing
Likely cause: DNS not updated or not propagated.

1. Click **update ip** again on DuckDNS; wait a minute.
2. `dig +short holdfast-ai.duckdns.org @1.1.1.1` (install with `sudo apt install -y dnsutils`).

Symptom: Health OK on the server but your browser can't open the site
Likely cause: Your computer cached old DNS.

1. Try a private window or another network; flush DNS (macOS: `sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder`).

Symptom: Want a real domain later
Likely cause:

1. Add an A record (e.g. `app` → static IP) at your registrar, set `DOMAIN`, `up -d`. Nothing else changes.

5. Running it: updates, logs, backups, scaling

5.1 Deploying a new version

```
server
cd ~/customer-intelligence && git pull
docker compose -f deploy/docker-compose.yml --env-file deploy/.env up -d --build
```

Only images whose files changed are rebuilt; the data volume is untouched. If a release adds database migrations for accounts, also run the `prisma migrate deploy` line from Step 7.

IF THIS BREAKS

Symptom: `git pull`: *Your local changes would be overwritten*

Likely cause: Files were edited on the server.

1. `git status` to see them; `git stash` (keeps them aside) then pull.
2. `deploy/.env` is ignored by git and is never affected.

5.2 Everyday commands

Task	Command
Status	`docker compose -f deploy/docker-compose.yml --env-file deploy/.env ps`
Follow logs	`docker compose -f deploy/docker-compose.yml --env-file deploy/.env logs -f api` (or `backend`, `web`)
Last 100 lines of all	`docker compose -f deploy/docker-compose.yml --env-file deploy/.env logs --tail=100`
Restart one service	`docker compose -f deploy/docker-compose.yml --env-file deploy/.env restart api`
Apply a changed `.env`	`docker compose -f deploy/docker-compose.yml --env-file deploy/.env up -d`
Stop everything (data kept)	`docker compose -f deploy/docker-compose.yml --env-file deploy/.env down`
Shell inside the API	`docker compose -f deploy/docker-compose.yml --env-file deploy/.env exec api bash`
Disk usage	`df -h` and `docker system df`
Memory	`free -h` and `docker stats --no-stream`

Never run `docker compose down -v` or `docker volume prune` unless you mean to delete all customer data: `-v` removes the named volumes.

5.3 Backups

Simplest: Lightsail → instance → **Snapshots** → **Automatic snapshots: on**. Daily full-disk copies (Docker volumes included), billed per GB actually used. Restore by creating a new instance from a snapshot.

Off-server copies with `deploy/backup.sh` (S3 or Cloudflare R2):

deploy/backup.sh

```
#!/bin/sh
# Nightly backup of the app data volume to an S3-compatible bucket (R2 or S3).
# Install on the server: sudo snap install aws-cli --classic, then add to crontab:
#   15 3 * * * /home/ubuntu/customer-intelligence/deploy/backup.sh >> /home/ubuntu/ci-backup.log
2>&1
set -eu
cd "$(dirname "$0")"
# Export the settings so the aws command sees the backup credentials.
set -a
. ./.env
set +a
stamp=$(date -u +%Y%m%dT%H%M%SZ)
archive="/tmp/ci-data-$stamp.tar.gz"
# A consistent copy: SQLite files are copied while writes are paused by a short stop of the api.
docker compose -f docker-compose.yml --env-file .env stop api
docker run --rm -v customer-intelligence_app-data:/data -v /tmp:/out alpine \
  tar czf "/out/$(basename "$archive")" -C /data .
docker compose -f docker-compose.yml --env-file .env start api
aws s3 cp "$archive" "s3://$BACKUP_BUCKET/ci-data/$(basename "$archive")" \
  ${BACKUP_ENDPOINT:+--endpoint-url "$BACKUP_ENDPOINT"}
rm -f "$archive"
echo "backup $stamp done"
```

server

```
sudo snap install aws-cli --classic
nano deploy/.env        # BACKUP_BUCKET, BACKUP_ENDPOINT (R2 only), AWS_ACCESS_KEY_ID,
AWS_SECRET_ACCESS_KEY
deploy/backup.sh        # run once by hand to test
crontab -e              # add the line below
15 3 * * * /home/ubuntu/customer-intelligence/deploy/backup.sh >> /home/ubuntu/ci-backup.log 2>&1
```

Restore (test this once):

server

```
docker compose -f deploy/docker-compose.yml --env-file deploy/.env down
docker run --rm -v customer-intelligence_app-data:/data -v /tmp:/in alpine \
  sh -c "rm -rf /data/* && tar xzf /in/ci-data-<stamp>.tar.gz -C /data"
docker compose -f deploy/docker-compose.yml --env-file deploy/.env up -d
```

IF THIS BREAKS

Symptom: `aws: command not found`
Likely cause: CLI not installed (apt has no `awscli` on 24.04).

1. `sudo snap install aws-cli --classic`.

Symptom: `Unable to locate credentials`
Likely cause: Keys empty, or an old script without `set -a`.

1. Fill the `AWS_*` lines; `git pull` for the fixed script.

Symptom: API stays stopped after a failed backup
Likely cause: The script stops the API while copying.

1. `docker compose -f deploy/docker-compose.yml --env-file deploy/.env start api`.

5.4 Upgrading the server size

1. Lightsail → instance → **Snapshots** → **Create snapshot**.
2. From the snapshot, **Create new instance** on the 4 GB plan.
3. Move the static IP to the new instance (Networking → detach/attach). DNS keeps working.
4. Delete the old instance when the new one is verified.

5.5 Rotating secrets

Secret	What happens if you change it
`BETTER_AUTH_SECRET`	Everyone is signed out; accounts are unaffected.
`INTERNAL_API_SECRET`	Nothing visible (both services read the same file); restart with `up -d`.
`CI_CREDENTIALS_KEY`	Saved connector credentials can no longer be decrypted: reconnect each connector.
Neon password	Update `DATABASE_URL`, then `up -d`.
Nebius / Resend keys	Update the value, then `up -d`.

5.6 Costs and housekeeping

- Create an AWS **budget alert** (Billing → Budgets) at a small amount so you know when free months/credits end. The free plan ends 6 months after sign-up.
- Static IPs are free only while attached: release unused ones.
- To shut down: delete the instance, release the static IP, delete snapshots you no longer need.
- The account looked like a managed/sandbox account (`assumed-role/AccountFullAccessRole`): confirm how long it lasts; the same steps work on your own account.

6. Alternatives and when to use them

Option	Works?	Notes
Own domain instead of DuckDNS	Yes	Best for clients and email sending (Resend needs a verified domain). A record → static IP, set `DOMAIN`, `up -d`.
Whole app on Vercel, backend on AWS	No	Cross-site cookies are blocked by browsers, or, if Vercel proxies `/api`, its 30-second proxy limit cuts streamed AI answers, briefs and large uploads.
Marketing pages on Vercel, app on AWS	Yes	Landing pages served from Vercel; "Sign in" links to the AWS app. Needs a marketing-only build.
Render / Railway	Mostly	Would need a persistent disk shared by API and worker (or splitting storage first); free tiers sleep, which stops the daily scheduler.
EC2 instead of Lightsail	Yes	Same Docker steps; more networking setup (security groups, Elastic IP).
Bigger scale	Planned	Move metadata from SQLite to PostgreSQL and files to S3/R2, then run several API/worker containers behind the same Caddy or a load balancer.

7. Appendix: cheat sheet and troubleshooting index

Full deploy in one place

server

```
# On a fresh Ubuntu 24.04 Lightsail instance (2 GB+), after the launch script ran:
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519 && cat ~/.ssh/id_ed25519.pub   # add as GitHub
deploy key
git clone git@github.com:itzzfalcon52/holdfast.git customer-intelligence && cd customer-
intelligence
cp deploy/.env.production.example deploy/.env && chmod 600 deploy/.env
sed -i "s|^DOMAIN=.*|DOMAIN=holdfast-ai.duckdns.org|" deploy/.env
sed -i "s|^BETTER_AUTH_SECRET=.*|BETTER_AUTH_SECRET=$(openssl rand -hex 32)|" deploy/.env
sed -i "s|^INTERNAL_API_SECRET=.*|INTERNAL_API_SECRET=$(openssl rand -hex 32)|" deploy/.env
K=$(openssl rand -base64 32 | tr '+/' '-_'); sed -i
"s|^CI_CREDENTIALS_KEY=.*|CI_CREDENTIALS_KEY=$K|" deploy/.env; unset K
nano deploy/.env    # DATABASE_URL, DIRECT_DATABASE_URL, NEBIUS_API_KEY
docker compose -f deploy/docker-compose.yml --env-file deploy/.env up -d --build
docker compose -f deploy/docker-compose.yml --env-file deploy/.env run --rm backend
./node_modules/.bin/prisma migrate deploy
docker compose -f deploy/docker-compose.yml --env-file deploy/.env ps
curl -s https://holdfast-ai.duckdns.org/api/v1/health
```

Troubleshooting index

Symptom	Where to look
Docker permission denied	Step 3
Build killed / exit 137 / no space	Step 6
Lockfile or pip hash errors	Step 6
git clone permission denied	Step 4
Prisma can't reach database	Step 7
Certificate / HTTPS errors, rate limits	Step 7
502 Bad Gateway	Step 7
503 account service unavailable	Step 8
403 origin not allowed / login doesn't stick	Step 8
Worker offline	Step 8
DNS not updated	Step 9
Backups failing	Section 5.3

Verification record

- Images built on the server in about 4½ minutes (api, web, backend) on the first attempt.
- Containers started; migrations applied; health check returned OK; app reachable over HTTPS.

- The deploy files were syntax-checked locally but first built on the server (Docker was not installed on the development Mac).